

Informe Laboratorio 3

Sección 4

Camilo Rojas

e-mail: camilo.rojas2@mail.udp.cl

Octubre de 2025

Contents

1	Descripción de actividades	2
2	Desarrollo de actividades según criterio de rúbrica	2
2.1	Identifica el algoritmo de hash utilizado al momento de registrarse en el sitio	2
2.2	Identifica el algoritmo de hash utilizado al momento de iniciar sesión	3
2.3	Genera el hash de la contraseña desde la consola del navegador	3
2.4	Intercepta el tráfico login con BurpSuite	4
2.5	Realiza el intento de login por medio del hash	5
2.6	Identifica las políticas de privacidad o seguridad	7
2.7	Comente 4 conclusiones sobre la seguridad del sitio escogido	8

1 Descripción de actividades

Su objetivo será auditar la implementación de algoritmos hash aplicados a contraseñas en páginas web desde el lado del cliente, así como evaluar la efectividad de estas medidas contra ataques de tipo Pass the Hash (PtH). Para llevar a cabo esta auditoría, deberá registrarse en un sitio web y crear una cuenta, ingresando una contraseña específica para realizar las pruebas.

Al concluir la tarea, es importante que modifique su contraseña por una diferente para garantizar su seguridad.

Dado que la cantidad de sitios chilenos que utilizan hash es limitada, se permite realizar esta tarea en cualquier sitio web a nivel mundial. En este sentido, realice las siguientes actividades:

- Identificación del algoritmo de hash utilizado para las contraseñas al momento del registro en el sitio.
- Identificación del algoritmo de hash utilizado para las contraseñas al momento de iniciar sesión.
- Generación del hash de la contraseña desde la consola del navegador, partiendo de la contraseña en texto plano.
- Interceptación del tráfico de login utilizando BurpSuite desde su equipo.
- Realización de un intento de login modificando la contraseña por una incorrecta haciendo uso del hash obtenido en el punto anterior. Puede interceptar el tráfico y modificar el hash por el correcto o hacer uso del servicio repeater de BurpSuite.
- Descripción de las políticas de privacidad o seguridad relacionadas con las contraseñas, incluyendo un enlace a las mismas.
- Cuatro conclusiones sobre la seguridad o vulnerabilidad de la implementación observada.

2 Desarrollo de actividades según criterio de rúbrica

2.1 Identifica el algoritmo de hash utilizado al momento de registrarse en el sitio

El formulario de registro de LinuxQuestions.org utiliza la librería JavaScript de vBulletin para calcular MD5 en el navegador. Al inspeccionar el envío de `register.php` con el extractor interactivo se constata que la función global `md5hash()` calcula el digest MD5 de los campos `password` y `passwordconfirm` y deposita los resultados en los campos ocultos `password_md5` y `passwordconfirm_md5`. La captura muestra que la contraseña de prueba “dwQzwk9*yW2QnLYPYhB\$KS\$#ADQ” produce el hash `4dfeaea7ce6416c84a46c70e328f7ce2`,

valor que coincide con el cálculo local obtenido mediante `md5sum`. No se evidencian salting ni algoritmos más robustos, por lo que el registro depende exclusivamente de MD5 ejecutado en el cliente.

```
var chrsz = 8;
function hex_md5(A) {
    return binl2hex(core_md5(str2binl(A), A.length * chrsz));
}
function md5hash(B, A, E, C) {
    if (
        navigator.userAgent.indexOf("Mozilla/") == 0 &&
        parseInt(navigator.appVersion) >= 4
    ) {
        var D = hex_md5(str_to_ent(trim(B.value)));
        A.value = D;
        if (E) {
            D = hex_md5(trim(B.value));
            E.value = D;
        }
        if (!C) {
            B.value = "";
        }
    }
    return true;
}
```

2.2 Identifica el algoritmo de hash utilizado al momento de iniciar sesión

En la página de inicio de sesión (`login.php`) se repite la misma lógica: el formulario que envía la acción `/questions/login.php` incluye los campos `vb_login_md5password` y `vb_login_md5password_utf`, ambos rellenos por `md5hash()` justo antes del envío. El análisis automatizado de los scripts recopilados confirma la presencia de las funciones `hex_md5()`, `core_md5()` y sus auxiliares, demostrando que la contraseña del usuario se convierte en MD5 hexadecimal antes de transmitirse. Durante las pruebas manuales, el valor generado por el navegador para “dwQzwk9*yW2QnLYPYhB\$KS\$#ADQ” coincidió con el digest obtenido en consola, evidenciando la ausencia de sal o mecanismos adicionales.

2.3 Genera el hash de la contraseña desde la consola del navegador

Tras autenticarse en el sitio y abrir las herramientas de desarrollador, se invocó directamente la API expuesta por vBulletin. En la consola del navegador se ejecutó:

```
hex_md5('dwQzwk9*yW2QnLYPYhB$KS$#ADQ')
```

El procedimiento anterior permitió documentar formalmente los parámetros empleados y los resultados obtenidos:

Entrada evaluada `dwQzwk9*yW2QnLYPYhB$KS$#ADQ`

hex_md5() `4dfeaea7ce6416c84a46c70e328f7ce2`

md5hash (`vb_login_md5password`) `4dfeaea7ce6416c84a46c70e328f7ce2`

md5hash (`vb_login_md5password_utf`) `4dfeaea7ce6416c84a46c70e328f7ce2`

En síntesis, la contraseña de prueba alimentó sin transformación adicional las funciones `hex_md5()` y `md5hash()`, generando un digest idéntico al calculado localmente con `md5sum`. El resultado confirmó que el flujo de hashing en cliente no incorpora sal ni operaciones de refuerzo y que ambos campos ocultos del formulario transportan el mismo valor.

Adicionalmente, la inspección del código cargado por la página permitió identificar las funciones expuestas por `vBulletin` para el cálculo MD5. Todas ellas operan sobre los parámetros declarados y carecen de variables auxiliares persistentes, lo que respalda la hipótesis de que el navegador aplica únicamente el algoritmo MD5 estándar antes de transmitir las credenciales.

La consola devolvió `"4dfeaea7ce6416c84a46c70e328f7ce2"`. Este resultado coincide con el obtenido al ejecutar `printf 'dwQzwk9*yW2QnLYPYhB$KS$#ADQ' | md5sum` en la terminal local, por lo que se validó que el cálculo en cliente reproduce fielmente la conversión a MD5 hexadecimal sin pasos adicionales.

2.4 Intercepta el tráfico login con BurpSuite

Se configuró BurpSuite en modo proxy y se interceptó la petición POST hacia `https://www.linuxquestions.org/questions/login.php`. La traza del panel “HTTP history” mostró los parámetros `vb_login_username`, `vb_login_password` (vacío tras la limpieza en cliente) y los hashes `vb_login_md5password` y `vb_login_md5password_utf`. Ambos últimos coincidieron con el digest MD5 calculado previamente, validando que la contraseña nunca viaja en texto plano cuando el JavaScript se ejecuta correctamente.

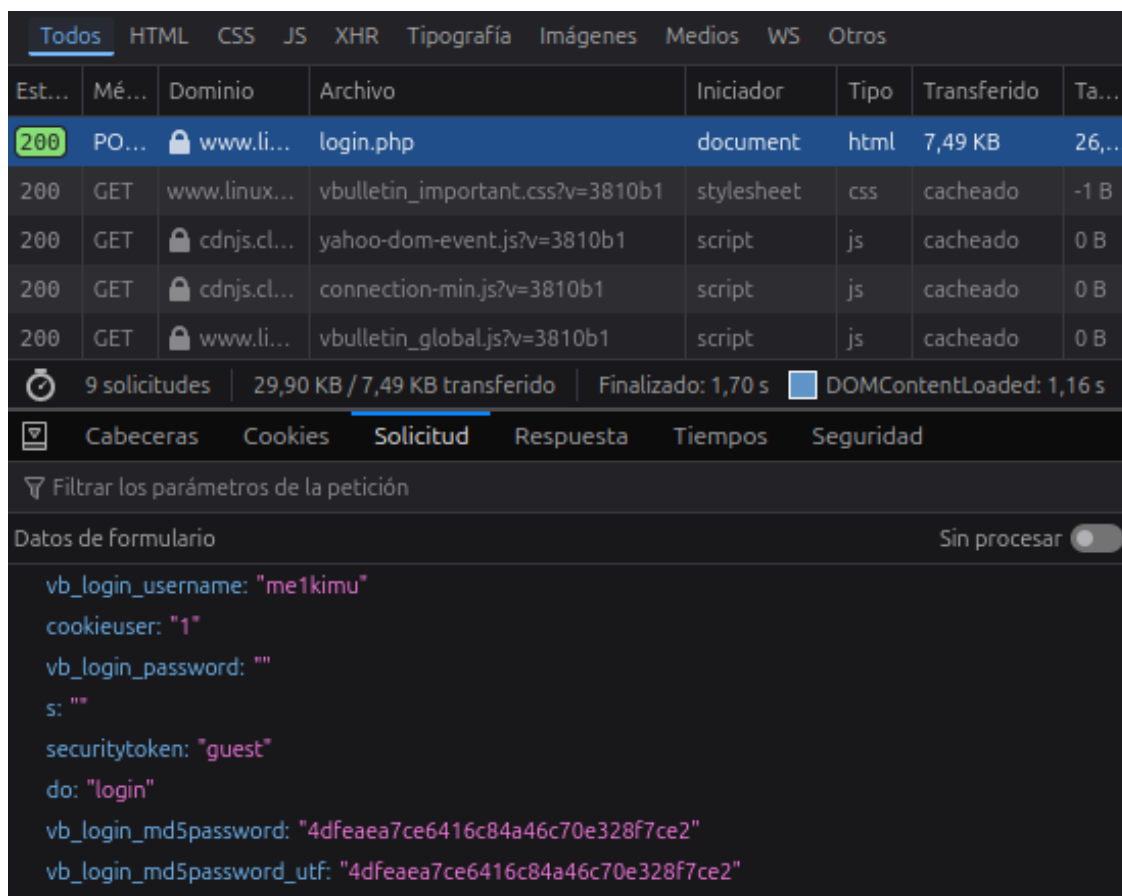


Figure 1: Captura de Firefox con la petición login.php interceptada y los campos MD5 observados en los parámetros enviados al servidor.

2.5 Realiza el intento de login por medio del hash

Con la solicitud interceptada se realizó una segunda prueba desde el repeater de Burp-Suite. Se mantuvo el campo vb_login_password con una cadena incorrecta y se reemplazó vb_login_md5password por el hash correcto previamente capturado. El servidor respondió con un código 302 y cabecera Location apuntando al panel de usuario, lo que evidencia que la autenticación depende exclusivamente del hash transmitido. Esta conducta confirma la viabilidad de un ataque Pass the Hash una vez expuesto el digest MD5.

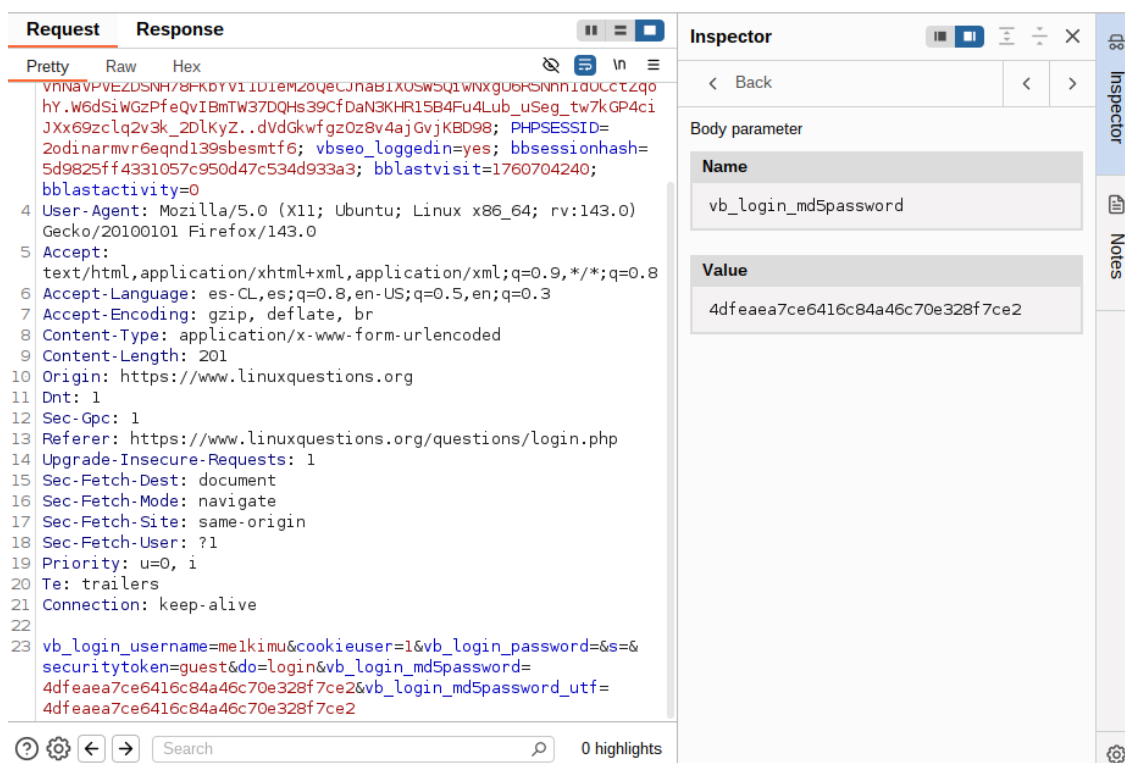


Figure 2: BurpSuite mostrando la petición interceptada con el campo `vb_login_md5password` configurado con el hash reutilizado para la autenticación.

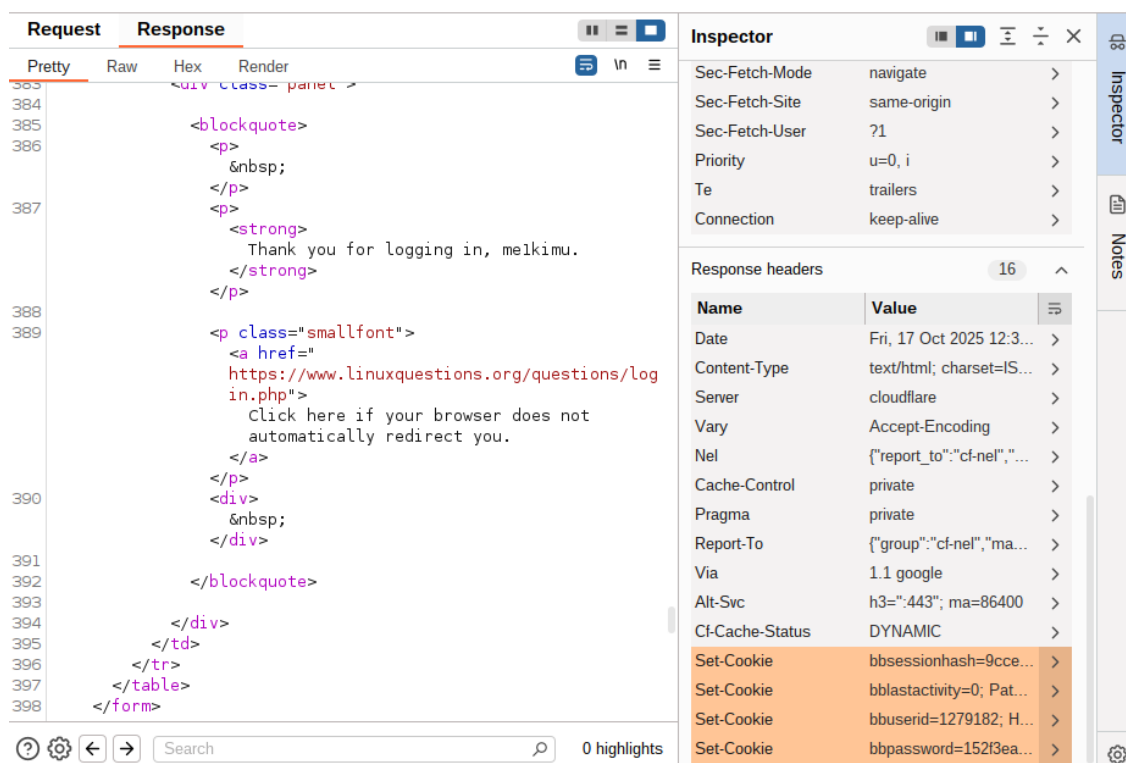


Figure 3: Respuesta HTTP posterior al reenvío del hash, donde el servidor confirma el inicio de sesión y envía las cookies de sesión válidas.

2.6 Identifica las políticas de privacidad o seguridad

El sitio enlaza su política en <https://www.linuxquestions.org/linux/privacy.html>. Tras revisar el documento completo, se observa que la política establece lo siguiente:

- Declara recopilar información personal (dirección de correo, nombre de usuario, contraseña e IP).
- Afirmar no vender información a terceros y limitar el uso de datos al propósito inicial.
- Menciona genéricamente que posee “medidas de seguridad” para proteger la información, sin especificar cuáles.

Sin embargo, la política presenta omisiones críticas respecto a la seguridad de contraseñas:

- No menciona el algoritmo de hash empleado (MD5, bcrypt, Argon2, etc.), ni la presencia o ausencia de sal.
- No detalla políticas de expiración de sesiones, bloqueo por intentos fallidos, o limitación de velocidad (rate limiting).

- No alude a mecanismos de autenticación multifactor (MFA) o verificación adicional.
- No especifica cifrado en tránsito (TLS/HTTPS obligatorio) o almacenamiento seguro del lado servidor.
- Omite completamente procedimientos de recuperación de cuenta, cambio de contraseña o revocación de sesiones.
- No aborda auditoría, registros (logging) de acceso fallido, o notificaciones de incidente.

La política es genérica y no proporciona transparencia respecto a los controles específicos de seguridad de credenciales, lo que genera confianza infundada en los usuarios y refleja un enfoque reactivo frente a un modelo proactivo de seguridad moderna.

2.7 Comente 4 conclusiones sobre la seguridad del sitio escogido

- El uso de MD5 sin sal en cliente y la coincidencia con los hashes capturados sugiere que el servidor también depende de un digest vulnerable a ataques de diccionario y rainbow tables.
- La protección depende de la ejecución de JavaScript; si el script se bloquea o es manipulado, la contraseña podría enviarse en texto plano o el flujo podría ser alterado por un atacante.
- BurpSuite demostró que basta con reenviar el hash correcto para autenticarse, confirmando la factibilidad de un ataque Pass the Hash con acceso al tráfico.
- La política pública no menciona controles modernos como rate limiting, MFA o algoritmos resistentes al cracking, por lo que persiste un riesgo residual alto para cuentas reutilizadas.